

Code:

```
#include <stdio.h> #include <stdlib.h>

typedef struct { long long u, v; } Pair;

int cmp_ll(const void *a, const void *b) { long long x = (long long)a; long long y = (long long)b; if (x < y) return -1; if (x > y) return 1; return 0; }

int cmp_pair(const void *a, const void *b) { Pair p1 = (Pair)a; Pair p2 = (Pair)b; if (p1->u != p2->u) return (p1->u < p2->u) ? -1 : 1; if (p1->v != p2->v) return (p1->v < p2->v) ? -1 : 1; return 0; }

int main() { int n; if (scanf("%d", &n) != 1) return 0;

long long *arr = (long long*)malloc(n * sizeof(long long));
long long max = -1;
int i;

for (i = 0; i < n; i++) {
    scanf("%lld", &arr[i]);
    if (arr[i] > max) {
        max = arr[i];
    }
}

// Generate and count unique ordered pairs (arr[i], arr[j]) with i < j
// Pre-coordinate compression
long long *temp = (long long*)malloc(n * sizeof(long long));
for (i = 0; i < n; i++) temp[i] = arr[i];
qsort(temp, n, sizeof(long long), cmp_ll);

int unq = 0;
for (i = 0; i < n; i++) {
    if (i == 0 || temp[i] != temp[i - 1]) {
        temp[unq++] = temp[i];
    }
}

int *disc = (int*)malloc(n * sizeof(int));
for (i = 0; i < n; i++) {
    int l = 0, r = unq - 1, pos = 0;
```

```

        while (l <= r) {
            int mid = (l + r) / 2;
            if (temp[mid] >= arr[i]) {
                pos = mid;
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }
        disc[i] = pos;
    }

// Track seen transitions using adjacency
char *seen = (char*)calloc((long long)unq * unq, sizeof(char));
int *first_seen = (int*)malloc(unq * sizeof(int));
for (i = 0; i < unq; i++) first_seen[i] = -1;

long long unique_pairs = 0;
for (i = 0; i < n; i++) {
    int u = disc[i];
    for (int k = 0; k < unq; k++) {
        if (first_seen[k] != -1 && !seen[(long long)k * unq + u]) {
            seen[(long long)k * unq + u] = 1;
            unique_pairs++;
        }
    }
    first_seen[u] = i;
}

printf("%lld\n", unique_pairs);

free(arr);
free(temp);
free(disc);
free(seen);
free(first_seen);
return 0;

}

```

Input:

5
1 2 2 1 3

Output:

6